70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · MT4

Docker va Kubernetes platformalaridan foydalanib sun'iy intellekt modelini joylashtirish (Deployment) va AI xizmatlarini boshqarish bo'yicha amaliy loyiha

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/MT4
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/MT4
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu mustaqil ishda siz bitta AI xizmatini (tumor-api — o'sma belgilarini tasniflovchi model) boshidan oxirigacha joylashtirish loyihasini bajarasiz: FastAPI xizmati va avtomatik testlar, qatlamli konfiguratsiya va sog'liq tekshiruvlari, Dockerfile va docker-compose generatsiyasi va validatsiyasi, dev/prod Kubernetes manifestlari (Deployment, Service, HPA, ConfigMap, Secret, Ingress, PDB, NetworkPolicy), rolling update va canary, HPA va Cluster Autoscaler yuklama simulyatsiyasi hamda SLO asosidagi operatsion hisobot. Sandbox'da Docker va Kubernetes yo'q: xizmat jarayon ichida TestClient bilan ishlaydi, fayllar matn sifatida yaratilib Python'da tekshiriladi, klaster xulq-atvori esa simulyatsiya qilinadi. Vaqt, narx va yuklama raqamlari taxminiy.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/MT4`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichda karkas (`s1_*.py`) berilgan: avval yo‘naltiruvchi savollarga o‘zingiz javob bering, so‘ng **TODO** joylarini to‘ldirib, `python3 s1_*.py` bilan ishga tushiring. Qiynalsangiz — qo‘llanma oxiridagi «Ishoralar» ilovasiga qarang. Natijani «Mening natijam» joyiga yozing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → MT4 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. AI xizmati: FastAPI, model reestri, API kalit, probe'lar va avtomatik testlar
2. 2-qadam. Konfiguratsiya qatlamlari, chuqur readiness, probe'lar va to'g'ri to'xtash
3. 3-qadam. Dockerfile va docker-compose: generatsiya, validatsiya va qatlam keshi
4. 4-qadam. Kubernetes manifestlari: generator, dev/prod overlay va 18 qoidali tekshiruv
5. 5-qadam. Versiyani chiqarish: RollingUpdate simulyatsiyasi va haqiqiy so'rovlar bilan canary
6. 6-qadam. HPA (autoscaling/v2) + Cluster Autoscaler + Erlang-C: yuklama simulyatsiyasi
7. 7-qadam. Operatsion hisobot: SLO, xato byudjeti, burn-rate ogohlantirishlari va Go/No-Go
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. AI xizmati: FastAPI, model reestri, API kalit, probe'lar va avtomatik testlar

MAQSAD

Loyiha — «tumor-api»: sklearn breast_cancer (30 belgi) bo'yicha o'sma turini tasniflovchi xizmat. Ikki model versiyasi (v1 — logistik regressiya, v2 — tasodifiy o'rmon) o'qitilib, SHA-256 xeshi bilan reestrga (/tmp/models) yoziladi. Xizmat `lifespan` ichida modelni butunlik tekshiruvi va isitish bilan yuklaydi; /startupz, /livez, /readyz alohida, /v1/predict va /v1/predict/batch API kalit (x-api-key) talab qiladi, har javobga x-request-id va x-model-version sarlavhalari qo'shiladi, /metrics Prometheus matn formatida. Oxirida CI'dagi kabi 11 ta avtomatik test TestClient bilan bajariladi. MODEL_VERSION, MAX_BATCH ni o'zgartirib ko'ring.

> **Kutilgan natija:** v1 aniqligi ~0,96, v2 ~0,94; 11/11 PASS; ketma-ket ~300-400 so'rov/s; /metrics da predictions_total 133 va model_info.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Model reestriga faqat model faylini yozish yetarlimi? Xizmat yuklash paytida fayl buzilmagan va aynan kerakli versiya ekanini qanday bilib oladi?

2. Nega model har so'rovda emas, `lifespan` ichida bir marta yuklanadi? Yuklagandan keyin bitta «bo'sh» bashorat (isitish) nima uchun kerak?

3. /startupz, /livez va /readyz probe'lari Kubernetes uchun nimasi bilan farq qiladi? Qaysi biri 503 qaytarishi mumkin va qachon?

4. Har javobga x-request-id ni qanday qo'shasiz: mijoz yuborgan ID bo'lsa nima qilasiz, bo'lmasa-chi? Bu qayerda — har endpoint ichidami yoki bitta joyda?

5. 29 ta belgi yuborilganda 422 qaytishi uchun tekshiruvni qayerga qo'yasiz? API kalitsiz so'rov 401, katta paket esa 413 bo'lishi uchun-chi?

6. Prometheus matn formatida counter va gauge qanday yoziladi? Testda predictions_total nega aynan 133 bo'lishi kerak?

7. Yakunda nimani kutasiz: v1 aniqligi ~0,96, v2 ~0,94, testlar 11/11 PASS va ~300–400 so'rov/s? FAIL chiqqan test qaysi TODO ga ishora qiladi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S1_*.PY)

```
# 1-qadam. AI xizmati: FastAPI, model reestri (v1/v2), API kalit, so'rov ID, batch, probe'lar
va avtomatik testlar
# Loyiha: "tumor-api" — ko'krak bezi o'smasi belgilarini (sklearn breast_cancer, 30 belgi)
tasniflovchi xizmat.
import warnings; warnings.filterwarnings("ignore")
import hashlib, json, os, pickle, time, uuid
from contextlib import asynccontextmanager
from typing import List

import numpy as np
from fastapi import FastAPI, Header, HTTPException, Request
from fastapi.responses import PlainTextResponse
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field, field_validator
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
REESTR = "/tmp/models"
for ver, est in (("v1", make_pipeline(StandardScaler(), LogisticRegression(max_iter=500))),
                 ("v2", RandomForestClassifier(n_estimators=60, random_state=0))):
    est.fit(Xtr, ytr); os.makedirs(f"{REESTR}/{ver}", exist_ok=True)
    # TODO 1: modelni pickle qilib model.pkl ga yozing; meta.json ga versiya, aniqlik va
sha256[:16] ni saqlang

MODEL_VERSION = os.getenv("MODEL_VERSION", "v1")
API_KEY = os.getenv("API_KEY", "maxfiy-demo-kalit")
MAX_BATCH = int(os.getenv("MAX_BATCH", "32"))
S = {"model": None, "meta": None, "startup": False, "ready": False, "n": 0, "xato": 0, "lat":
[], "boshlangan": time.time()}


@asynccontextmanager
async def lifespan(app):
    # TODO 2: meta va modelni o'qing, sha256 bilan butunligini tekshiring, isiting (warm-up),
startup/ready ni yoqing
    raise NotImplementedError("TODO 2: lifespan")
    yield


app = FastAPI(title="tumor-api", version="1.0.0", lifespan=lifespan)

# TODO 3: middleware — x-request-id (kelganini saqlash yoki 12 belgili yangi) va x-model-
version sarlavhalari, /v1/ kechikishi


class Namuna(BaseModel):
    belgilar: List[float] = Field(..., description="30 ta son belgisi")
    # TODO 4: field_validator — aynan 30 ta va faqat chekli (NaN/inf emas) qiymatlar


class Paket(BaseModel):
    namunalar: List[Namuna]


def kalit(x_api_key):
```

```
    # TODO 5: kalit noto'g'ri bo'lsa xato hisoblagichini oshirib, 401 qaytaring
    raise NotImplementedError("TODO 5")

# TODO 6: /startupz, /livez (uptime_s), /readyz (tayyor bo'lmasa 503), /version yo'llarini
yozing
# TODO 7: /v1/predict (sinf: "xavfli"/"xavfsiz", ehtimol) va /v1/predict/batch (MAX_BATCH dan
ko'p -> 413)
# TODO 8: /metrics — predictions_total, auth_errors_total, p95 kechikish va model_info
Prometheus matn formatida

# --- Avtomatik test to'plami (o'zgartirmang) ---
K = {"x-api-key": API_KEY}
natija = []
def test(nom, shart):
    natija.append((nom, "PASS" if shart else "FAIL"))

with TestClient(app) as c:
    test("startupz 200", c.get("/startupz").json()["ok"])
    test("readyz 200", c.get("/readyz").status_code == 200)
    r = c.post("/v1/predict", json={"belgilar": Xte[0].tolist()}, headers=K)
    test("predict 200 + sinf", r.status_code == 200 and r.json()["sinf"] in ("xavfli",
"xavfsiz"))
    test("x-request-id sarlavhasi", len(r.headers.get("x-request-id", "")) == 12)
    test("so'rov ID saqlanadi", c.get("/livez", headers={"x-request-id": "abc123"}).headers["x-
request-id"] == "abc123")
    test("API kalitsiz -> 401", c.post("/v1/predict", json={"belgilar":
Xte[0].tolist()}).status_code == 401)
    test("29 belgi -> 422", c.post("/v1/predict", json={"belgilar": [0.0] * 29},
headers=K).status_code == 422)
    test("batch 33 -> 413", c.post("/v1/predict/batch", json={"namunalar": [{"belgilar": [0.0]
* 30}] * 33}, headers=K).status_code == 413)
    rb = c.post("/v1/predict/batch", json={"namunalar": [{"belgilar": x.tolist()} for x in
Xte[:32]]}, headers=K)
    test("batch 32 aniqlik >= 0.9", rb.status_code == 200 and np.mean(np.array(rb.json()
["natija"]) == yte[:32]) >= 0.9)
    test("/version sha256", c.get("/version").json()["sha256"] == S["meta"]["sha256"])
    t0 = time.perf_counter()
    for x in Xte[:100]:
        c.post("/v1/predict", json={"belgilar": x.tolist()}, headers=K)
    rps = 100 / (time.perf_counter() - t0)
    m = c.get("/metrics").text
    test("metrics: predictions_total", "predictions_total 133" in m)

print("\n=== tumor-api testlari ===")
for nom, h in natija:
    print(f"{h:4s}  {nom}")
print(f"Jami: {sum(h == 'PASS' for _, h in natija)}/{len(natija)} PASS;  ketma-ket ~{rps:.0f}
so'rov/s (TestClient, 1 jarayon)")
print("\n--- /metrics ---\n" + m)
print("OpenAPI yo'llari:", ", ".join(sorted(app.openapi()["paths"])))
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== tumor-api testlari ===`, `Jami: 11/11 PASS`, `predictions_total`, `OpenAPI yo'llari`

TAJRIBA QILIB KO‘RING

- MODEL_VERSION ni os.environ orqali 'v2' qiling — /version va x-model-version sarlavhasi o'zgarishini tekshiring.
- Model faylini yozib bo'lgach bir baytini o'zgartiring: xizmat ishga tushmasligi (butunlik tekshiruvi) kerak.

- /metrics ga model bo'yicha so'rovlar sonini (label bilan) qo'shing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Konfiguratsiya qatlamlari, chuqur readiness, probe'lar va to'g'ri to'xtash

MAQSAD

(a) Sozlamalar uch qatlamdan yig'iladi: standart < ConfigMap (YAML) < muhit o'zgaruvchilari (TUMOR_*); har kalit sxema bo'yicha tur va diapazonga tekshiriladi, sir maskalanadi, xato bo'lsa xizmat ishga tushmaydi (fail fast); konfiguratsiya xeshi Deployment annotatsiyasi uchun hisoblanadi. (b) /readyz model fayli, uning xeshi, disk, xotira, o'z-o'zini sinash va drain holatini tekshiradi; /livez esa bog'liqliklarni tekshirmaydi. (c) Kubelet probe'lari simulyatsiyasi: model 45 s yuklanganda noto'g'ri liveness yoki qisqa startupProbe CrashLoopBackOff'ga olib kelishi ko'rsatiladi. (d) preStop va terminationGracePeriod ning yo'qolgan so'rovlarga ta'siri.

> **Kutilgan natija:** To'g'ri konfiguratsiyada OK va config-hash, xatoda 3 ta xato; buzilgan faylda readyz 503, livez 200; 2 ta XATO konfiguratsiyada 6 restart va hech qachon tayyor emas, to'g'rilarida 45 s; preStop=0 da ~300 so'rov yo'qoladi.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Bitta kalit uch joyda (standart, ConfigMap, muhit o'zgaruvchisi) berilgan bo'lsa, qaysi biri g'olib bo'lishi kerak va nega aynan shu tartib?

2. Muhit o'zgaruvchilari doim satr bo'lib keladi. "128" ni int ga o'tkazish va "ikki" ni xato deb topish uchun SXEMA dan qanday foydalanasiz?

3. Nega xato konfiguratsiyada xizmat umuman ishga tushmasligi (fail fast) yaxshiroq? Konfiguratsiya xeshini Deployment annotatsiyasiga qo'yish nima beradi?

4. Readiness va liveness probe'lari nimani tekshirishi kerak? Model fayli buzilganda /livez ham 503 qaytarsa nima bo'ladi?

5. Model 45 s yuklansa, liveness 10 s kechikish bilan 5 s periodda 3 marta tekshirilsa nima bo'ladi? startupProbe 2 s × 15 bilan-chi?

6. preStop = 0 bo'lsa, nega Pod to'xtatilganda so'rovlar yo'qoladi? Endpoint yangilanishi 3 s kechiksa va 100 so'rov/s kelsa, taxminan nechta?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S2_*.PY)

```
# 2-qadam. Konfiguratsiya qatlamlari va sog'liq tekshiruvlari: sozlamalar validatsiyasi, chuqur
readiness,
# kubelet probe'lari simulyatsiyasi (CrashLoopBackOff) va to'g'ri to'xtash (graceful shutdown)
import warnings; warnings.filterwarnings("ignore")
import hashlib, json, os, shutil, time
import psutil, yaml
from fastapi import FastAPI
from fastapi.responses import JSONResponse
from fastapi.testclient import TestClient

# --- Berilgan ma'lumotlar (o'zgartirmang) ---
STANDART = {"model_version": "v1", "max_batch": 32, "log_level": "INFO", "workers": 2,
"timeout_s": 5.0, "api_key": ""}
SXEMA = {  # kalit: (tur, tekshiruv, izoh)
    "model_version": (str, lambda v: v in ("v1", "v2", "v3"), "v1|v2|v3"),
    "max_batch": (int, lambda v: 1 <= v <= 256, "1..256"),
    "log_level": (str, lambda v: v in ("DEBUG", "INFO", "WARNING", "ERROR"), "log darajasi"),
    "workers": (int, lambda v: 1 <= v <= 8, "1..8 (CPU limitiga mos)"),
    "timeout_s": (float, lambda v: 0.1 <= v <= 60, "0,1..60 s"),
    "api_key": (str, lambda v: len(v) >= 16, ">= 16 belgi (Secret)"),
}
CONFIGMAP = """
model_version: v2
max_batch: 64
log_level: INFO
workers: 2
"""


def yukla(configmap_text, env):
    """Qaytaradi: (cfg, manba, xatolar)."""
    # TODO 1: standart -> ConfigMap (yaml.safe_load) -> TUMOR_<KALIT> muhit o'zgaruvchilari
tartibida birlashtiring, manbani yozing
    # TODO 2: SXEMA bo'yicha har kalitni turga o'tkazing va diapazonini tekshiring; xatolarni
ro'yxatga yig'ing
    raise NotImplementedError("TODO 1-2: konfiguratsiya yuklash")


def maskala(cfg):
    # TODO 3: api_key ni '***' + oxirgi 3 belgi ko'rinishida yashiring
    raise NotImplementedError("TODO 3")


print("=== (a) Konfiguratsiya qatlamlari ===")
for nom, env in (("to'g'ri", {"TUMOR_API_KEY": "s3cr3t-kalit-2026-xyz", "TUMOR_MAX_BATCH":
"128"}),
                 ("xato", {"TUMOR_API_KEY": "qisqa", "TUMOR_WORKERS": "ikki",
"TUMOR_TIMEOUT_S": "120"})):
    cfg, manba, xato = yukla(CONFIGMAP, env)
    # TODO 4: maskalangan sozlama va manbalarni chiqaring; xato bo'lsa "fail fast", bo'lmasa
config-hash (sha256, sort_keys)

os.makedirs("/tmp/models/v2", exist_ok=True)
open("/tmp/models/v2/model.pkl", "wb").write(b"demo-model-bytes")
KUTILGAN = hashlib.sha256(b"demo-model-bytes").hexdigest()
app = FastAPI()
HOLAT = {"draining": False}


def tekshiruvlar():
    # TODO 5: model fayli, uning sha256, disk (>50 MB), RSS xotira (<1200 MB), self-test (<50
ms) va drain holatini tekshiring
    raise NotImplementedError("TODO 5: chuqur readiness tekshiruvlari")
```

```
# TODO 6: /livez (faqat jarayon tirikmi) va /readyz (hammasi True bo'lsa 200, aks holda 503 + tekshiruvlar) yo'llari

print("\n=== (b) Chuqur readiness ===")
# TODO 7: TestClient bilan normal, buzilgan fayl, livez va drain holatlarini sinang


def kubelet(startup_s, probe, T=600):
    """Konteyner startup_s soniyada tayyor bo'ladi. liveness/startup ketma-ket failureThreshold marta yiqilsa -> restart;
    kutish (backoff) 10, 20, 40 ... 300 s. Qaytaradi: (tayyor_vaqt yoki None, restartlar, voqealar)."""
    # TODO 8: startupProbe (period*threshold) yoki liveness (delay + period*i) ilova tayyor bo'lgunga qadar yiqiladimi — modellang
    raise NotImplementedError("TODO 8: kubelet simulyatsiyasi")


KONF = {  # liv: (period, failureThreshold), rdy: readiness period
    "XATO: liveness 10s, startupProbe yo'q": {"liv_delay": 10, "liv": (5, 3), "rdy": 5},
    "XATO: startupProbe 2s x 15 (30 s)":    {"startup": (2, 15), "liv": (10, 3), "rdy": 5},
    "TO'G'RI: startupProbe 2s x 30":         {"startup": (2, 30), "liv": (10, 3), "rdy": 5},
    "TO'G'RI: liveness delay 60s":           {"liv_delay": 60, "liv": (10, 3), "rdy": 5},
}
print("\n=== (c) Kubelet probe simulyatsiyasi: model yuklanishi 45 s ===")
for nom, pr in KONF.items():
    tayyor, rs, v = kubelet(45, pr)
    print(f"\n{nom}: tayyor = {tayyor if tayyor is not None else 'HECH QACHON (600 s ichida)'}, restartlar = {rs}")

print("\n=== (d) To'xtashda yo'qolgan so'rovlar (100 so'rov/s, so'rov 0,4 s gacha) ===")
# TODO 9: preStop/grace juftliklari (0,30), (5,30), (5,3), (10,30) uchun yo'qolgan so'rovlarni baholang (endpoint kechikishi 3 s)
print("\n=== Konfiguratsiya va sog'liq tekshiruvlari tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== (a) Konfiguratsiya qatlamlari ===`, `=== (b) Chuqur readiness ===`, `=== (c) Kubelet probe simulyatsiyasi`, `=== Konfiguratsiya va sog'liq tekshiruvlari tugadi ===`

TAJRIBA QILIB KO‘RING

- Sxemaga 'redis_url' (redis:// bilan boshlanishi shart) kalitini qo'shing va xato holatni sinang.
- Model yuklanishini 90 s qiling — startupProbe parametrlarini qanday tanlaysiz?
- Endpoint yangilanish kechikishini 3 s dan 8 s ga oshiring: preStop qancha bo'lishi kerak?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Dockerfile va docker-compose: generatsiya, validatsiya va qatlam keshi

MAQSAD

Loyiha parametrlaridan ko'p bosqichli (multi-stage) Dockerfile generatsiya qilinadi: builder bosqichida wheel'lar, yakuniy obrazda faqat runtime, root bo'lmagan foydalanuvchi, HEALTHCHECK va exec-shakldagi CMD. 13 qoidali tahlilchi uni ataylab yomon yozilgan Dockerfile bilan solishtiradi. Qatlam keshi xeshlar zanjiri bilan simulyatsiya qilinadi: faqat kod o'zgarganda nechta qatlam qayta qurilishi va yomon tartibda pip ham qayta ishlashi ko'rsatiladi. So'ng API + Redis + Prometheus uchun docker-compose.yml pyyaml bilan yaratiladi va tekshiriladi: obraz teglari, port to'qnashuvi, depends_on + healthcheck, hajm va tarmoqlar e'loni, sirlar, ishga tushish tartibi (topologik saralash) va sikl.

> **Kutilgan natija:** Yaxshi Dockerfile 13/13, yomoni 0/13; kod o'zgarganda 1 qatlam (~3 s), requirements o'zgarganda 5 qatlam (~200 s, taxminiy); compose OK, tartib redis -> api -> prometheus; buzilgan compose'da 4 xato.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Nega Dockerfile ikki bosqichli (builder va yakuniy) qilinadi? Yakuniy obrazga builder'dan nima o'tadi va nima o'tmaydi?

2. Xavfsizlik nuqtai nazaridan yakuniy obrazda qaysi uch narsa bo'lishi shart: kim nomidan ishlaydi, sirlar qayerda, sog'liq qanday tekshiriladi?

3. 13 ta qoidani matndan tekshirish uchun Dockerfile ni qanday bo'laklarga ajratasiz? Masalan, «ENV da sir yo'q» ni qanday regex bilan topasiz?

4. Docker qatlam keshi qanday ishlaydi: qachon qatlam qayta quriladi? Nega requirements.txt ni koddan oldin alohida COPY qilish kerak?

5. Faqat app/main.py o'zgarganda yaxshi Dockerfile da nechta qatlam qayta qurilishini kutasiz? requirements o'zgarganda-chi? Yomon Dockerfile da pip nega qayta ishlaydi?

6. Compose faylda xizmatlar ishga tushish tartibi depends_on dan qanday chiqariladi va sikl bo'lsa buni qanday aniqlaysiz?

7. Buzilgan compose'da (prometheus 8000 porti, redis healthcheck'siz, API_KEY environment'da, redis → prometheus bog'liqligi) qaysi 4 xato chiqishi kerak?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S3_*.PY)

```
# 3-qadam. Dockerfile va docker-compose: shablondan generatsiya, qoidalar bo'yicha validatsiya
va qatlam keshi simulyatsiyasi
# Docker sandbox'da yo'q — fayllar matn sifatida yaratiladi va Python'da tahlil qilinadi
(hadolint/compose config o'rniga).
import hashlib, re
import yaml

LOYIHA = {"nom": "tumor-api", "python": "3.12-slim-bookworm", "port": 8000, "uid": 10001,
          "paketlar": ["fastapi==0.115.6", "uvicorn==0.34.0", "scikit-learn==1.6.1",
"numpy==2.2.1"]}


def dockerfile(p):
    """Ko'p bosqichli (multi-stage) Dockerfile: builder'da g'ildiraklar (wheels), yakuniy
obrazda faqat runtime."""
    # TODO 1: builder (requirements -> pip wheel) va yakuniy bosqich (non-root USER, COPY --
from, HEALTHCHECK, exec-CMD) matnini qaytaring
    raise NotImplementedError("TODO 1: Dockerfile shabloni")


YOMON = """FROM python:latest
ENV API_KEY=maxfiy123
COPY . .
RUN pip install fastapi uvicorn scikit-learn
RUN apt-get update
RUN apt-get install -y curl
EXPOSE 8080
CMD uvicorn app.main:app --port 8000
"""

QOIDALAR = ["D01 ko'p bosqichli (>=2 FROM)", "D02 bazaviy obraz teg bilan, ':latest' emas",
"D03 slim/distroless baza",
            "D04 USER (root emas)", "D05 ENV da sir yo'q", "D06 'COPY . .' yo'q", "D07
requirements koddan oldin (kesh)",
            "D08 pip --no-cache-dir", "D09 versiyalar qotirilgan (==)", "D10 HEALTHCHECK bor",
"D11 EXPOSE = ilova porti",
            "D12 CMD exec shaklida (JSON)", "D13 apt-get update alohida RUN emas"]


def tahlil(txt, port):
    """Qaytaradi: {qoida: True/False} — QOIDALAR ro'yxatidagi har biri uchun."""
    # TODO 2: izohsiz qatorlar, ko'rsatmalar va FROM larni ajrating; har qoidani satr/regex
tekshiruvi bilan yozing
    raise NotImplementedError("TODO 2: Dockerfile linter")


YAXSHI = dockerfile(LOYIHA)
print("=== Dockerfile qoidalari: yaxshi vs yomon ===")
# TODO 3: ikkala Dockerfile uchun PASS/FAIL jadvali va ballni chiqaring


def qurish(txt, fayllar, kesh):
    """Har ko'rsatma xeshi = oldingi xesh + ko'rsatma + tegishli fayllar xeshi. Qaytaradi:
qayta qurilgan RUN/COPY qatlamlar."""
    # TODO 4: FROM yangi zanjir boshlaydi; COPY ga fayllar mazmuni, COPY --from ga o'sha
bosqich xeshi qo'shiladi; keshda yo'q xeshlar qayta quriladi
    raise NotImplementedError("TODO 4: qatlam keshi")


VAQT = {"RUN pip": 95, "RUN groupadd": 2}           # taxminiy qatlam qurish vaqti, s
(qolganlari 3 s)
```

```
fayllar = {"requirements.txt": "fastapi==0.115.6...", "app/main.py": "v1-kod", "app/model.py":
"m"}
print("\n=== Qatlam keshi (multi-stage, requirements alohida) ===")
# TODO 5: avval to'liq qurib keshni to'ldiring; keyin faqat kod va faqat requirements
o'zgarganda nechta qatlam (va necha s) qayta qurilishini chiqaring; YOMON bilan solishtiring

COMPOSE = {
    "services": {
        "api": {"build": ".", "image": "registry.local/tumor-api:1.0.0", "ports":
["8000:8000"],
                "environment": {"MODEL_VERSION": "v1", "REDIS_URL": "redis://redis:6379/0"},
                "env_file": [".env"], "volumes": ["models:/models:ro"], "read_only": True,
                "depends_on": {"redis": {"condition": "service_healthy"}},
                "healthcheck": {"test": ["CMD", "python", "-c", "import
urllib.request;urllib.request.urlopen('http://127.0.0.1:8000/readyz')"],
                                "interval": "15s", "timeout": "3s", "retries": 3,
"start_period": "40s"},
                "deploy": {"resources": {"limits": {"cpus": "2", "memory": "1g"}}}, "restart":
"unless-stopped",
                "networks": ["ichki"]},
        "redis": {"image": "redis:7.4-alpine", "healthcheck": {"test": ["CMD", "redis-cli",
"ping"], "interval": "5s", "retries": 5},
                  "networks": ["ichki"], "restart": "unless-stopped"},
        "prometheus": {"image": "prom/prometheus:v3.1.0", "ports": ["9090:9090"], "volumes":
["./prometheus.yml:/etc/prometheus/prometheus.yml:ro"],
                       "depends_on": {"api": {"condition": "service_healthy"}}, "networks":
["ichki"]},
    },
    "volumes": {"models": {}},
    "networks": {"ichki": {}},
}
matn = yaml.safe_dump(COMPOSE, sort_keys=False, allow_unicode=True)


def compose_tekshir(c):
    """Qaytaradi: (xatolar ro'yxati, ishga tushish tartibi)."""
    # TODO 6: obraz tegi, xost port to'qnashuvi, depends_on + healthcheck, hajm/tarmoq e'loni,
environment ichidagi sirlarni tekshiring
    # TODO 7: depends_on bo'yicha topologik saralash va sikl aniqlash
    raise NotImplementedError("TODO 6-7: compose validatsiya")


x, tartib = compose_tekshir(yaml.safe_load(matn))
print("Compose validatsiya:", "OK" if not x else x, "| ishga tushish tartibi:", " ->
".join(tartib))
# TODO 8: compose'ni ataylab buzing (port, healthcheck, sir, sikl) va 4 xato chiqishini
ko'rsating
print("\n=== Dockerfile va compose tekshiruvi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Dockerfile qoidalari`, `=== Qatlam keshi`, `Compose validatsiya: OK`, `=== Dockerfile va compose tekshiruvi tugadi ===`

TAJRIBA QILIB KO‘RING

- Dockerfile'ga .dockerignore generatsiyasini va uni tekshiruvchi qoidani qo'shing (.git, __pycache__, *.ipynb).
- Compose'ga GPU ishlatadigan 'trainer' xizmatini (deploy.resources.reservations.devices) qo'shing va validatorni kengaytiring.
- Yomon Dockerfile'ni bosqichma-bosqich tuzatib, har tuzatishdan keyin ballni yozib boring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Kubernetes manifestlari: generator, dev/prod overlay va 18 qoidali tekshiruv

MAQSAD

Python generatori bitta parametrlar lug'atidan 9 ta obyekt yaratadi: Namespace, ConfigMap, Secret, Deployment (startup/readiness/liveness probe, preStop, securityContext, topologySpread, checksum/config annotatsiyasi, NAS'dagi PVC), Service, HPA (autoscaling/v2), PodDisruptionBudget, Ingress (TLS) va NetworkPolicy. Kustomize'ga o'xshash chuqur birlashtirish bilan dev va prod overlay'lari olinadi, natija multi-hujjat YAML'ga yozilib qayta o'qiladi. Tekshiruvchi obyekt qoidalari bilan birga obyektlararo moslikni ham ko'radi: Service selector, targetPort, Ingress backend, HPA target, PDB, configMapKeyRef kalitlari. Ataylab buzilgan manifestda tipik xatolar ushlanadi.

> **Kutilgan natija:** dev 8 obyekt (1 nusxada PDB yo'q), prod 9 obyekt; dev va prod 18/18 PASS, buzilgan 11/18; Ingress YAML ko'rsatiladi.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. dev va prod muhitlari bir-biridan nimasi bilan farq qiladi va nega har biri uchun alohida YAML yozmasdan, bitta asos + overlay ishlatiladi?

2. Deployment da Pod shabloni label'lari, selector va Service selector qanday bog'langan? Qaysi biri mos kelmasa trafik Pod'larga yetmaydi?

3. ConfigMap o'zgarganda Pod'lar avtomatik qayta yaratilishi uchun nima qilinadi? Konteynerga sozlama va sir qanday yo'llar bilan beriladi?

4. 1 nusxali dev muhitida PodDisruptionBudget nega zararli? Uni yaratish shartini qanday yozasiz va prod da minAvailable qancha bo'lishi kerak?

5. Obyektlararo tekshiruvlarni qulay yozish uchun hujjatlarni qanday indekslaysiz? Ingress backend porti Service portiga mosligini qanday tekshirasiz?

6. CPU "500m" va "1", xotira "512Mi" va "1Gi" ni qanday solishtirasiz? Buzilgan manifestda requests.cpu = "2" qaysi qoidani yiqitadi?

7. Natijani tekshiring: dev va prod 18/18 PASS, buzilgan manifest 11/18 chiqdimi? YAML'ga yozib qayta o'qish nega foydali?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S4_*.PY)

```
# 4-qadam. Kubernetes manifestlari: Python generatori (dev/prod overlay), pyyaml bilan multi-hujjat YAML,
# obyekt qoidalari va obyektlararo moslik tekshiruvi (Service selector, Ingress backend, HPA target, ConfigMap kalitlari)
import copy, hashlib, json
import yaml

ASOS = {"nom": "tumor-api", "ns": "ai-prod", "obraz": "registry.local/tumor-api:1.0.0", "port": 8000,
        "replicas": 3, "cpu_req": "500m", "cpu_lim": "1", "mem_req": "512Mi", "mem_lim": "1Gi",
        "hpa": (3, 10, 65), "host": "tumor.ai.local", "config": {"MODEL_VERSION": "v1", "MAX_BATCH": "32", "LOG_LEVEL": "INFO"}}
OVERLAY = {"dev": {"ns": "ai-dev", "replicas": 1, "hpa": (1, 2, 80), "host": "tumor-dev.ai.local", "config": {"LOG_LEVEL": "DEBUG"}},
           "prod": {}}


def birlashtir(a, b):
    """Kustomize'ga o'xshash chuqur birlashtirish (deep merge)."""
    # TODO 1: a ning nusxasiga b ni rekursiv qo'shing (ikkalasi dict bo'lsa — ichkariga, aks holda almashtirish)
    raise NotImplementedError("TODO 1: deep merge")


def generatsiya(p):
    """Qaytaradi: Namespace, ConfigMap, Secret, Deployment, Service, HPA, PDB, Ingress, NetworkPolicy lug'atlari ro'yxati."""
    # TODO 2: umumiy label'lar va config xeshi (checksum/config annotatsiyasi uchun)
    # TODO 3: Deployment — RollingUpdate, securityContext, topologySpread, envFrom/secretKeyRef/configMapKeyRef,
    #         resources, startup/readiness/liveness probe'lar, preStop, PVC (models-nas) ni yozing
    # TODO 4: Service (targetPort "http"), HPA autoscaling/v2, PDB, Ingress (TLS), NetworkPolicy obyektlarini qo'shing
    # TODO 5: replicas < 2 bo'lsa PDB ni chiqarib tashlang
    raise NotImplementedError("TODO 2-5: manifest generatori")


def mcpu(v):
    return int(v[:-1]) if str(v).endswith("m") else int(float(v) * 1000)


def mib(v):
    return int(v[:-2]) * (1024 if v.endswith("Gi") else 1)


QOIDALAR = ["K01 obraz tegi qotirilgan (latest emas)", "K02 requests/limits bor", "K03 requests <= limits",
            "K04 startup/readiness/liveness probe", "K05 readiness != liveness yo'li", "K06 runAsNonRoot + readOnlyRootFS",
            "K07 sir Secret orqali (ochiq env emas)", "K08 selector == template labels", "K09 Service selector Pod'larga mos",
            "K10 Service targetPort konteynerda bor", "K11 Ingress backend Service mavjud", "K12 Ingress TLS bor",
            "K13 HPA target Deployment mavjud", "K14 HPA min <= replicas <= max", "K15 PDB minAvailable < replicas (drain bloklanmaydi)",
            "K16 configMapKeyRef kalitlari mavjud", "K17 checksum/config annotatsiyasi", "K18 hamma obyekt bitta namespace'da"]


def tekshir(hujjatlar):
```

```
    """Qaytaradi: {qoida: bool} — QOIDALAR dagi 18 ta qoida uchun."""
    # TODO 6: (kind, name) -> obyekt lug'atini tuzing, Deployment va konteynerni toping; K01–
K08 obyekt qoidalarini yozing
    # TODO 7: K09–K18 obyektlararo moslik: Service/Ingress/HPA/PDB/ConfigMap havolalari
haqiqatan mavjudmi
    raise NotImplementedError("TODO 6-7: manifest tekshiruvchisi")


natija = {}
for muhit, ov in OVERLAY.items():
    p = birlashtir(ASOS, ov)
    docs = generatsiya(p)
    # TODO 8: safe_dump_all -> safe_load_all aylanma tekshiruvi; natija[muhit] = tekshir(...);
prod uchun Ingress YAML ni chiqaring
    PROD = docs

buz = copy.deepcopy(PROD)
d = next(h for h in buz if h["kind"] == "Deployment"); cc = d["spec"]["template"]["spec"]
["containers"][0]
cc["image"] = "registry.local/tumor-api:latest"; cc["resources"]["requests"]["cpu"] = "2"
cc["env"].append({"name": "API_KEY_OCHIQ", "value": "123"}); cc["env"][1]["valueFrom"]
["configMapKeyRef"]["key"] = "MODEL_VER"
next(h for h in buz if h["kind"] == "Service")["spec"]["selector"] = {"app": "tumor"}
next(h for h in buz if h["kind"] == "Ingress")["spec"]["rules"][0]["http"]["paths"][0]
["backend"]["service"]["port"] = {"number": 8080}
next(h for h in buz if h["kind"] == "PodDisruptionBudget")["spec"]["minAvailable"] = 3
natija["buzilgan"] = tekshir(buz)

print("=== Manifest qoidalari (dev / prod / buzilgan) ===")
for m, q in natija.items():
    print(f"{m:9s}: {sum(map(bool, q.values()))}/{len(q)} PASS")
print("\n=== K8s manifestlari tekshiruvi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Manifest qoidalari`, `prod : 18/18 PASS`, `=== K8s manifestlari tekshiruvi tugadi ===`

TAJRIBA QILIB KO‘RING

- 'staging' overlay qo'shing (replicas 2, boshqa host) va uchala muhitni bitta jadvalda solishtiring.
- HPA'ga ikkinchi metrika (Pods turidagi http_requests_per_second) qo'shing va validatorga uning tekshiruvini yozing.
- Generatsiya qilingan YAML'ni faylga yozib, real klasterda `kubectl apply --dry-run=server -f` bilan tekshiring (agar klaster bo'lsa).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Versiyani chiqarish: RollingUpdate simulyatsiyasi va haqiqiy so'rovlar bilan canary

MAQSAD

(a) Deployment kontrolleri 1 s qadamli modelda simulyatsiya qilinadi: 6 nusxa, har biri 50 so'rov/s, talab 260 so'rov/s, yangi Pod 30 s da tayyor. maxSurge/maxUnavailable kombinatsiyalari, blue/green va Recreate uchun yangilanish davomiyligi, minimal sig'im, yo'qolgan so'rovlar va maksimal Pod soni solishtiriladi. (b) Ikki FastAPI versiyasi TestClient orqali ishga tushadi va vaznli marshrutlovchi (Ingress/Service mesh o'rnida) trafikni 5 -> 25 -> 50 -> 100 % bosqichlarda bo'ladi. Har bosqichda kamida 60 canary namunasi yig'ilib, xato ulushi va p95 kechikish bazaviy versiya bilan solishtiriladi; mezon buzilsa avtomatik orqaga qaytariladi. Sog'lom, nuqsonli va sekin v2 sinab ko'riladi.

> **Kutilgan natija:** maxUnavailable=0 da yo'qotish 0, maxUnavailable=1 da ~1800, Recreate da ~7800 so'rov yo'qoladi; canary: sog'lom v2 100 % gacha chiqadi, nuqsonli va sekin v2 5 % bosqichda to'xtatiladi.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. 6 nusxa × 50 so'rov/s = 300 so'rov/s sig'im va talab 260. Yangilanish paytida bir vaqtda nechta Pod ishlamay qolsa so'rovlar yo'qola boshlaydi?

2. maxSurge va maxUnavailable Deployment kontrolleriga nimani ruxsat beradi? Har soniyada qaysi tartibda qaror qabul qilinadi?

3. Recreate strategiyasini qanday modellaysiz va unda nima uchun yo'qotish eng katta? Blue/green (surge=6) ning bahosi nimada?

4. Natijalaringiz kutilganga mosmi: maxUnavailable=0 da yo'qotish 0, maxUnavailable=1 da ~1800, Recreate da ~7800 so'rov?

5. Canary'da trafikni w % ehtimol bilan v2 ga qanday yo'naltirasiz? Nega har versiyadan kamida 60 namuna yig'ish kerak?

6. Avtomatik orqaga qaytarish mezoni qanday bo'lishi kerak? Nega p95 uchun nisbiy (1,5×) bilan birga mutlaq (+2 ms) chegara ham qo'yiladi?

7. Uch ssenariyda nimani kutasiz: sog'lom v2 100 % gacha chiqadimi, nuqsonli va sekin v2 qaysi bosqichda to'xtaydi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S5_*.PY)

```
# 5-qadam. Yangi versiyani chiqarish: RollingUpdate simulyatsiyasi (maxSurge/maxUnavailable) va
# haqiqiy so'rovlar bilan canary — ikki FastAPI versiyasi, vaznli marshrutlovchi (Ingress
o'rnida) va avtomatik tahlil
import warnings; warnings.filterwarnings("ignore")
import math, random, time
import numpy as np
import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel


def rolling(N=6, surge=1, unav=0, start_s=30, sig=50, talab=260, recreate=False, T=900):
    """1 s qadamli model. Har Pod sig' so'rov/s; yangi Pod start_s dan keyin tayyor.
    Qaytaradi: (davomiylik s, minimal sig'im, yo'qolgan so'rovlar, maksimal Pod soni)."""
    # TODO 1: har soniyada: start tugagan Pod'larni tayyorga o'tkazing; eski Pod'larni
mavjudlik >= N-unav saqlanguncha o'chiring
    # TODO 2: jami Pod <= N+surge bo'lguncha yangi Pod yarating (Recreate: avval hammasi
o'chadi, surge yo'q)
    # TODO 3: sig'im = tayyor Pod * sig; yo'qolgan += max(0, talab - sig'im); hammasi
yangilanganda natijani qaytaring
    raise NotImplementedError("TODO 1-3: RollingUpdate modeli")


print("=== (a) RollingUpdate: 6 nusxa x 50 so'rov/s, talab 260 so'rov/s, Pod start 30 s ===")
for nom, kw in (("maxSurge=1, maxUnavailable=0", dict(surge=1, unav=0)), ("maxSurge=2,
maxUnavailable=0", dict(surge=2, unav=0)),
                ("maxSurge=0, maxUnavailable=1", dict(surge=0, unav=1)), ("maxSurge=25%(2),
maxUnav=25%(2)", dict(surge=2, unav=2)),
                ("maxSurge=6 (blue/green kabi)", dict(surge=6, unav=0)), ("Recreate",
dict(recreate=True))):
    d, ms, tu, mp = rolling(**kw)
    print(f"{nom:34s} {d:12d} {ms:11d} {tu:17.0f} {mp:9d}")


def xizmat(versiya, xato_ulush, qoshimcha_ms):
    app = FastAPI(); rng = random.Random(sum(map(ord, versiya)))   # takrorlanuvchi urug'

    class Kirish(BaseModel):
        x: list[float]

    @app.post("/predict")
    def predict(k: Kirish):
        t0 = time.perf_counter()
        s = float(np.tanh(np.array(k.x) @ np.linspace(-1, 1, len(k.x))))
        while (time.perf_counter() - t0) * 1e3 < qoshimcha_ms: pass     # sun'iy qo'shimcha
hisob
        if rng.random() < xato_ulush: raise HTTPException(500, "ichki xato")
        return {"versiya": versiya, "ball": s}
    return TestClient(app)


def canary(v1, v2, qadamlar=(5, 25, 50, 100), min_namuna=60, maks_sorov=1500, seed=1):
    """Qaytaradi: har bosqich bo'yicha jurnal DataFrame (vazn_%, xato_v1_%, xato_v2_%,
p95_v1_ms, p95_v2_ms, qaror)."""
    rng = random.Random(seed); jurnal = []
    for w in qadamlar:
        # TODO 4: w % ehtimol bilan v2 ga, qolganini v1 ga yuboring; har versiyadan kamida
min_namuna (status, ms) yig'ing
        # TODO 5: xato ulushi (status >= 500) va p95 kechikishni hisoblang (100 % da baza —
oldingi bosqichdan)
```

```
        # TODO 6: xato > baza + 2 p.p. yoki p95 > max(1.5*baza, baza+2 ms) bo'lsa — "ORQAGA
QAYTARISH" va to'xtash
        raise NotImplementedError("TODO 4-6: canary tahlili")
    return pd.DataFrame(jurnal)


v1 = xizmat("v1", 0.005, 0.0)
for nom, v2 in (("v2 sog'lom (xato 0,5 %, +0 ms)", xizmat("v2", 0.005, 0.0)),
                ("v2 nuqsonli (xato 8 %)", xizmat("v2b", 0.08, 0.0)),
                ("v2 sekin (+6 ms hisob)", xizmat("v2c", 0.005, 6.0))):
    j = canary(v1, v2)
    print(f"\n=== (b) Canary: {nom} ===")
    print(j.to_string(index=False))
print("\n=== Rolling va canary tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== (a) RollingUpdate`, `=== (b) Canary: v2 nuqsonli`, `=== Rolling va canary tugadi ===`

TAJRIBA QILIB KO‘RING

- Canary qaroriga ikki ulush uchun z-test qo'shing ($p < 0,05$) va qat'iy chegara bilan solishtiring.
- Pod start vaqtini 90 s qiling: qaysi strategiya maqbul bo'ladi?
- v2 xato ulushini 2,5 % qiling — canary uni qaysi bosqichda va necha so'rovdan keyin ushlaydi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. HPA (autoscaling/v2) + Cluster Autoscaler + Erlang-C: yuklama simulyatsiyasi

MAQSAD

HPA'ning haqiqiy algoritmi modellashtiriladi: desired = ceil(current x metrika / nishon), 10 % tolerantlik, bir nechta metrikada maksimum, scaleUp siyosati (15 s da 100 % yoki 4 Pod) va scaleDown uchun 300 s stabilizatsiya oynasi. Pod'lar tugunlarga joylashadi (tugunda 3 Pod), joy yetmasa Pending bo'ladi va Cluster Autoscaler yangi tugun so'raydi (120 s), ortiqcha tugunni 10 daqiqadan keyin olib tashlaydi. Kechikish M/M/c (Erlang C) bilan baholanadi. 6 soatlik kunlik to'lqin va 10 daqiqalik keskin sakrash (+450 so'rov/s) ostida on-prem (4 tugun) va bulut konfiguratsiyalari SLO buzilish daqiqalari, yo'qolgan so'rovlar, Pod-soat va tugun-soat bo'yicha solishtiriladi.

> **Kutilgan natija:** On-prem 4 tugunda sakrashda ~10 daqiqa SLO buzilishi va ~110 ming so'rov yo'qoladi; bulut CA bilan ~1,5-5 daqiqa; Erlang-C: 300 so'rov/s da 7 Pod p95 ~110 ms, 10 Pod ~64 ms.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. HPA formulasi desired = ceil(current × metrika / nishon). Agar 4 Pod 90 % CPU da ishlasa va nishon 70 % bo'lsa, nechta Pod tavsiya qilinadi? Tolerantlik 10 % qachon o'zgarishni to'xtatadi?

2. Nega CPU metrikasi keskin sakrashda yomon ishlaydi, RPS metrikasi esa yaxshiroq? CPU foydalanishni qanday hisoblaysiz va u qayerda to'yinadi?

3. scaleUp va scaleDown siyosatlari nega nosimmetrik? 300 s stabilizatsiya oynasi qanday «tebranish» (flapping) ni oldini oladi?

4. Pod yaratildi, lekin tugunda joy yo'q — u qaysi holatda turadi? Cluster Autoscaler qachon yangi tugun so'raydi va qachon olib tashlaydi?

5. M/M/c navbatida Erlang C nimani bildiradi? Undan p95 javob vaqtini qanday baholaysiz?

6. Natijani tekshiring: Erlang-C da 300 so'rov/s, 7 Pod p95 ~110 ms, 10 Pod ~64 ms chiqdimi? On-prem 4 tugunda sakrashda ~10 daqiqa SLO buzilishi va ~110 ming so'rov yo'qolishi kutiladi — nega aynan shuncha?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S6_*.PY)

```
# 6-qadam. HPA (autoscaling/v2) algoritmi + Cluster Autoscaler + Erlang-C navbat modeli:
yuklama simulyatsiyasi
# Pod 1 yadro so'raydi, 1 so'rov = 20 ms CPU -> Pod sig'imi ~50 so'rov/s; tugun -> 3 Pod.
Kechikish — M/M/c (Erlang C).
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DT = 15                          # HPA sinxronlash davri, s
T = 6 * 3600                     # 6 soat
XIZMAT = 0.020                   # s/so'rov (CPU vaqti)
MU = 1 / XIZMAT                  # 50 so'rov/s/Pod
REQ_CPU = 1.0                    # Pod requests.cpu = limits.cpu = 1 yadro (Guaranteed QoS)
SLO_MS = 200


def yuklama(t):
    """Kunlik to'lqin (80 -> 320 so'rov/s) + t=3 soatda 10 daqiqalik keskin sakrash (aksiya,
+450)."""
    s = 80 + 240 * max(0.0, math.sin(math.pi * t / T)) ** 2
    if 3 * 3600 <= t < 3 * 3600 + 600: s += 450
    return s


def erlang_c(lam, c, mu):
    """M/M/c: p95 javob vaqti (ms); rho >= 1 (ortiqcha yuklama) da None."""
    # TODO 1: rho, Erlang C (navbatda kutish ehtimoli) va p95 = kutish + 3*XIZMAT ni hisoblang
    raise NotImplementedError("TODO 1: Erlang-C")


def sim(nom, min_r=2, max_r=30, cpu_nishon=70, rps_nishon=None, tugun_max=4, tugun_boshl=4,
tugun_start=120,
        pod_start=30, pod_per_tugun=3):
    """Qaytaradi: (natija lug'ati: konfiguratsiya, SLO_buzilish_daq, yo'qolgan_so'rov,
Pod_soat, tugun_soat, maks_Pod; log)."""
    for t in range(0, T, DT):
        lam = yuklama(t)
        # TODO 2: start tugagan Pod va tugunlarni tayyorga o'tkazing; Pending Pod'larni bo'sh
joyga joylang
        # TODO 3: HPA: desired = ceil(tayyor * metrika / nishon), 10 % tolerantlik, metrikalar
bo'yicha maksimum, [min_r, max_r]
        # TODO 4: scaleUp: 15 s da max(100 %, 4 Pod); scaleDown: 300 s oynadagi eng katta
tavsiya, 60 s da 10 %
        # TODO 5: Cluster Autoscaler: Pending bo'lsa tugun so'rash (tugun_start s), 10 daqiqa
ortiqcha tugunni olib tashlash
        # TODO 6: SLO (Erlang-C p95 > 200 ms yoki ortiqcha yuklama), yo'qolgan so'rovlar, Pod-
soat va tugun-soatni yig'ing
        raise NotImplementedError("TODO 2-6: HPA + CA simulyatsiyasi")


KONF = [
    ("asos: CPU 70 %, 4 tugun (on-prem)", {}),
    ("CPU 50 % (zaxira ko'p)", dict(cpu_nishon=50)),
    ("CPU 70 % + RPS 30/Pod", dict(rps_nishon=30)),
    ("bulut: CA 4 -> 10 tugun", dict(tugun_max=10)),
    ("bulut + RPS 30/Pod", dict(tugun_max=10, rps_nishon=30)),
    ("bulut + sekin tugun (300 s)", dict(tugun_max=10, tugun_start=300)),
    ("bulut + oldindan 6 tugun, min 8", dict(tugun_max=10, tugun_boshl=6, min_r=8)),
]
natija = []; loglar = {}
for nom, kw in KONF:
    r, lg = sim(nom, **kw); natija.append(r); loglar[nom] = lg
```

```
print("=== HPA + Cluster Autoscaler simulyatsiyasi (6 soat, 15 s qadam) ===")
print(pd.DataFrame(natija).to_string(index=False))

print("\n=== Erlang-C: 300 so'rov/s da Pod soniga qarab p95 (ms) ===")
# TODO 7: c = 7, 8, 9, 10, 12 uchun foydalanish va p95 ni chiqaring; sig'im va Pod soni
grafiklarini chizing
print("\n=== HPA simulyatsiyasi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== HPA + Cluster Autoscaler simulyatsiyasi`, `=== Erlang-C`, `=== HPA simulyatsiyasi tugadi ===`

TAJRIBA QILIB KO‘RING

- scaleUp siyosatini 15 s da 2 Pod ga cheklang — SLO buzilishi qanday o'zgaradi?
- Sakrashni prognoz qilib (aksiya jadvali), t=2:55 da minReplicas ni vaqtincha 16 ga ko'taring (KEDA cron kabi).
- Pod start vaqtini 30 s dan 10 s ga tushirsangiz (kichik obraz, model keshlangan) natija qancha yaxshilanadi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Operatsion hisobot: SLO, xato byudjeti, burn-rate ogohlantirishlari va Go/No-Go

MAQSAD

30 kunlik daqiqalik so'rov/xato qatori (kunlik to'lqin, 12 % o'sish, ikki insident: keskin va sekin) sintetik yaratiladi. SLO 99,9 % uchun xato byudjeti va uning sarfi, Google SRE Workbook'dagi ko'p oynali burn-rate qoidalari (1 soat/5 daq > 14,4x, 6 soat/30 daq > 6x, 3 kun/6 soat > 1x) hisoblanib, har insidentni aniqlash kechikishi topiladi. Shu qoidalar asosida PrometheusRule (monitoring.coreos.com/v1) YAML generatsiya qilinadi va validatsiya qilinadi. Oxirida kunlik cho'qqi trendi bo'yicha sig'im prognozi va oldingi qadamlar natijalarini jamlovchi Go/No-Go nazorat ro'yxati.

> **Kutilgan natija:** Mavjudlik ~99,92 %, byudjet sarfi ~84 %; keskin insident 5 daqiqada, sekin insident faqat 6 soatlik va 3 kunlik qoidalarda aniqlanadi; PrometheusRule VALID (5 qoida); sig'im ~45 kunga yetadi; QAROR: GO.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. SLO 99,9 % bo'lsa, 30 kunlik xato byudjeti nima va u qanday birlikda o'lchanadi? Mavjudlik ~99,92 % bo'lganda byudjetning qancha qismi sarflangan bo'ladi?

2. Burn rate 1x, 6x va 14,4x nimani anglatadi? 14,4x tezlikda 30 kunlik byudjet qancha vaqtda tugaydi?

3. 43200 daqiqalik qatorda 3 kunlik (4320 daq) sirpanuvchi oyna yig'indisini sikl bilan hisoblash juda sekin. Buni bir harakatda qanday qilasiz?

4. Nega har qoida ikki oynani (uzun VA qisqa) talab qiladi? Faqat uzun oyna bo'lsa insident tugagandan keyin nima bo'ladi?

5. Keskin (40 daq, 15 % xato) va sekin (12 soat, 1,2 % xato) insidentlarni qaysi qoidalar ushlashini kutasiz? Aniqlanish kechikishini qanday topasiz?

6. PrometheusRule YAML ni yaratgandan keyin uni qaysi xatolarga tekshirasiz? Nega «page» darajasidagi ogohlantirishda runbook_url majburiy?

7. Sig'im prognozida kunlik cho'qqining chiziqli trendini qanday topasiz va «necha kun qoldi» ni qanday hisoblaysiz? Go/No-Go qarori qachon GO bo'ladi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S7_*.PY)

```
# 7-qadam. Operatsion hisobot: SLO va xato byudjeti, ko'p oynali burn-rate ogohlantirishlari,
PrometheusRule YAML
# generatsiyasi va validatsiyasi, sig'im prognozi va yakuniy Go/No-Go nazorat ro'yxati.
import math
import numpy as np
import pandas as pd
import yaml
import matplotlib.pyplot as plt

# --- Berilgan ma'lumotlar: 30 kunlik daqiqalik so'rov/xato qatori (o'zgartirmang) ---
rng = np.random.default_rng(7)
DAQ = 30 * 24 * 60
t = np.arange(DAQ)
kun = (t % 1440) / 1440
sorov = (120 + 200 * np.sin(np.pi * kun) ** 2) * 60 * (1 + 0.004 * t / 1440)      #
so'rov/daqiqa, 30 kunda +12 % o'sish
sorov = rng.poisson(sorov).astype(float)
xato_ulush = np.full(DAQ, 0.0004)
INSIDENT = [("model fayli buzilgan (keskin)", 9 * 1440 + 600, 40, 0.15),
            ("xotira sizishi (sekin)", 21 * 1440 + 300, 720, 0.012)]
for nom, bosh, dav, e in INSIDENT:
    xato_ulush[bosh:bosh + dav] = e
xato = rng.binomial(sorov.astype(int), xato_ulush).astype(float)

SLO = 0.999
print("=== SLO va xato byudjeti (30 kun) ===")
# TODO 1: mavjudlik, byudjet = (1 - SLO) * jami so'rov, sarflangan % va har insidentning
byudjetdagi ulushini chiqaring
BYUDJET = None


def burn(oyna):
    """Oyna (daqiqa) bo'yicha har daqiqa uchun burn rate = xato ulushi / (1 - SLO)."""
    # TODO 2: kumulyativ yig'indilar (np.cumsum) bilan sirpanuvchi oynadagi xato ulushini tez
hisoblang
    raise NotImplementedError("TODO 2: burn rate")


# Google SRE Workbook tavsiyasi: (nom, uzun oyna daq, qisqa oyna daq, chegara, darajasi)
QOIDA = [("XatoByudjetTez", 60, 5, 14.4, "page"), ("XatoByudjetOrta", 360, 30, 6.0, "page"),
("XatoByudjetSekin", 4320, 360, 1.0, "ticket")]
print("\n=== Ko'p oynali burn-rate ogohlantirishlari ===")
yongan = {}
# TODO 3: har qoida uchun (uzun > ch) & (qisqa > ch) niqobini yongan ga yozing; har
insidentning aniqlanish kechikishini (daq) toping


def ifoda(oyna):
    # TODO 4: oyna uchun 5xx xato ulushining PromQL ifodasini qaytaring
(rate(http_requests_total[...]))
    raise NotImplementedError("TODO 4: PromQL")


# TODO 5: monitoring.coreos.com/v1 PrometheusRule lug'atini tuzing: 3 ta burn-rate +
PodQaytaIshgaTushmoqda + HPAMaksimumda
rule = None
matn = yaml.safe_dump(rule, sort_keys=False, allow_unicode=True, width=200)


def rule_tekshir(r):
    """Qaytaradi: (xatolar, qoidalar soni)."""
```

```
    # TODO 6: kind, majburiy maydonlar, severity, qavslar, 'for' formati, page uchun
runbook_url, takroriy nomlarni tekshiring
    raise NotImplementedError("TODO 6: PrometheusRule validatsiya")


xat, n = rule_tekshir(yaml.safe_load(matn))
print(f"\n=== PrometheusRule: {n} qoida, {len(matn.splitlines())} qator YAML -> {'VALID' if not
xat else xat} ===")

# TODO 7: kunlik cho'qqining chiziqli trendi (np.polyfit) bo'yicha 12 Pod x 50 x 70 % sig'im
necha kunga yetishini toping
kun_qoldi = None

ROYXAT = [
    ("Xizmat testlari 11/11 PASS (1-qadam)", True),
    ("Konfiguratsiya validatsiyasi fail-fast (2-qadam)", True),
    ("startupProbe bilan CrashLoop yo'q (2-qadam)", True),
    ("Dockerfile 13/13, compose OK (3-qadam)", True),
    ("K8s manifest 18/18 prod (4-qadam)", True),
    ("Rollout: maxUnavailable=0, canary avtomatik orqaga qaytaradi (5-qadam)", True),
    ("HPA/CA: sakrashda SLO buzilishi <= 5 daq (6-qadam, bulut CA)", True),
    # TODO 8: mavjudlik >= SLO, byudjet < 100 %, qoidalar VALID va insidentlar aniqlangan,
sig'im >= 30 kun bandlarini qo'shing
]
print("\n=== Go/No-Go nazorat ro'yxati ===")
# TODO 9: bandlarni [+]/[-] bilan chiqaring va yakuniy QAROR (GO / NO-GO) ni chiqaring
print("\n=== Operatsion hisobot tayyor ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== SLO va xato byudjeti`, `=== Ko'p oynali burn-rate ogohlantirishlari ===`, `VALID`, `=== Go/No-Go nazorat ro'yxati ===`, `=== Operatsion hisobot tayyor ===`

TAJRIBA QILIB KO‘RING

- SLO ni 99,95 % ga oshiring — qaror va qaysi bandlar o'zgaradi?
- Kechikish SLO'si (p95 < 200 ms) uchun ham burn-rate qoidasini qo'shing (histogram_quantile ifodasi bilan).
- Insidentlar sonini 4 taga oshirib, byudjet tugaydigan kunni va 'muzlatish' (feature freeze) siyosatini hisobotga yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Ilova. Ishoralar

Avval savolga o‘zingiz javob berishga harakat qiling; ishorani faqat qiynalganda o‘qing. To‘liq namuna yechim saytdagi virtual laboratoriyada bosqichni bajarganingizdan keyin ochiladi.

1. 1-qadam. AI xizmati: FastAPI, model reestri, API kalit, probe'lar va avtomatik testlar

1. meta.json ga versiya, aniqlik va pickle baytlarining SHA-256 xeshini (dastlabki 16 belgi) yozing; lifespan da xeshni qayta hisoblab solishtiring.

```
hashlib.sha256(blob).hexdigest()[:16]
```

2. Startup — og'ir ish bir marta. Isitish birinchi foydalanuvchi so'rovi sekin bo'lmasligi uchun. yield dan keyingi kod — SIGTERM da drain (ready=False).
3. livez — jarayon tirikmi (har doim ok); readyz — trafik qabul qila oladimi (model yuklanmagan bo'lsa 503); startupz — ishga tushish tugadimi.
4. @app.middleware("http") — bitta joyda. Kelgan sarlavhani saqlang, aks holda uuid4().hex[:12]. /v1/ yo'llari kechikishini S["lat"] ga yozing.

```
rid = request.headers.get("x-request-id") or uuid.uuid4().hex[:12]
```

5. Pydantic field_validator ValueError chiqarsa FastAPI o'zi 422 beradi. Kalit — Header(None) parametri va HTTPException(401); paket uzunligi > MAX_BATCH → HTTPException(413).
6. `# TYPE nom counter` qatori, keyin `nom qiymat`. Hisoblang: 1 predict + 32 batch + 100 ketma-ket = 133 (401 va 422 so'rovlar hisoblanmaydi).
7. Har test bitta talabni tekshiradi — nomidan qaysi endpoint yoki sarlavha ekanini toping. MODEL_VERSION=v2 bilan ham ishga tushirib ko'ring.

Bajargach o‘ylang: Namunadagi middleware va kalit() funksiyasini o'zingiznikiga solishtiring: API kalitni Depends() orqali tekshirish qanday afzallik beradi? Nega xato hisoblagichi (auth_errors_total) foydali?

2. 2-qadam. Konfiguratsiya qatlamlari, chuqur readiness, probe'lar va to'g'ri to'xtash

1. Ustuvorlik: standart < ConfigMap < env. Env — eng aniq (Pod darajasida, Secret'dan). Har kalit qayerdan kelganini manba lug'atida saqlang.
2. tur(qiymat) ni try/except (TypeError, ValueError) ichida chaqiring, keyin shart(qiymat) ni tekshiring. Xatoli konfiguratsiyada 3 ta xato chiqishi kerak.

```
cfg[k] = tur(cfg[k])
```

3. Xato sozlama bilan ishlagan xizmat jimgina noto'g'ri natija beradi. Xesh o'zgarsa Pod shabloni o'zgaradi va Pod'lar qayta yaratiladi. json.dumps(..., sort_keys=True) — barqaror xesh uchun.
4. readyz — bog'liqliklar (model, xesh, disk, xotira, drain); livez — faqat jarayon. Buzilgan faylda livez yiqilsa, kubelet keraksiz restart qiladi — bu muammoni hal qilmaydi.

5. Ilova tayyor bo'lgunga qadar threshold marta yiqilsa — restart va backoff (10, 20, 40… 300 s). 2×15 = 30 s < 45 s, demak har safar yiqiladi. 2 xato konfiguratsiyada 6 restart, to'g'rilarida tayyor 45 s.
6. Trafik hali kelayotgan paytda jarayon yopiladi: 100 × (3 − preStop) so'rov. grace < preStop + 0,4 bo'lsa, ishlanayotgan so'rovlar ham uziladi. preStop=0 da ~300 kutiladi.

Bajargach o‘ylang: Namunadagi kubelet() modelini o'zingiznikiga solishtiring. Real Kubernetes'da yana qaysi omil (masalan, readiness yiqilganda Service'dan chiqarish) bu modelda hisobga olinmagan?

3. 3-qadam. Dockerfile va docker-compose: generatsiya, validatsiya va qatlam keshi

1. Builder'da pip wheel bilan g'ildiraklar yasaladi; yakuniy obrazga faqat /wheels (COPY --from=builder) o'tadi, kompilyator va kesh qolmaydi. Baza — slim, teg qotirilgan.
2. USER (uid 10001, root emas), ENV da KEY/SECRET yo'q (Secret orqali), HEALTHCHECK /readyz ga. CMD — JSON (exec) shaklida, aks holda SIGTERM ilovaga yetmaydi.
3. Bo'sh va # bilan boshlanuvchi qatorlarni tashlang; har qatorning birinchi so'zi — ko'rsatma. Yaxshi fayl 13/13, yomoni 0/13 bo'lishi kerak.

```
re.search(r"ENV.*(KEY|SECRET|PASSWORD|TOKEN)=", l, re.I)
```

4. Har qatlam xeshi = oldingi xesh + ko'rsatma + nusxalangan fayllar mazmuni. Bitta qatlam o'zgarsa, undan keyingilarning hammasi qayta quriladi.

```
h = hashlib.sha1((h + l + manba).encode()).hexdigest()
```

5. Kutilgan: kod — 1 qatlam (~3 s), requirements — 5 qatlam (~200 s). YOMON da COPY . . birinchi turadi.
6. Topologik saralash: har qadamda barcha bog'liqliklari allaqachon tartibda bo'lgan xizmatlarni oling; hech biri tayyor bo'lmasa — sikl. Kutilgan: redis → api → prometheus.
7. Port to'qnashuvi, service_healthy shartida healthcheck yo'qligi, environment'dagi sir va depends_on sikli.

Bajargach o‘ylang: Namunadagi tahlil() qoidalarini o'zingiznikiga solishtiring: qaysi qoidani regex bilan yozish ishonchsiz (yolg'on PASS beradi)? Haqiqiy loyihada hadolint qanday afzallik beradi?

4. 4-qadam. Kubernetes manifestlari: generator, dev/prod overlay va 18 qoidali tekshiruv

1. Faqat ns, replicas, hpa, host va LOG_LEVEL farq qiladi. Chuqur birlashtirish (deep merge) config ichidagi bitta kalitni almashtiradi, qolganlarini saqlaydi — oddiy dict.update esa butun config ni yo'qotadi.
2. selector.matchLabels ⊆ template.metadata.labels bo'lishi shart; Service.spec.selector ham shu label'larni tanlashi kerak (K08, K09).
3. Pod shabloniga checksum/config annotatsiyasi (config xeshi) qo'ying. Sozlama — envFrom configMapRef, sir — secretKeyRef (ochiq value emas).

```
hashlib.sha256(json.dumps(p["config"], sort_keys=True).encode()).hexdigest()[:10]
```

4. minAvailable ≥ replicas bo'lsa tugun drain'i butunlay bloklanadi. replicas ≥ 2 da yarating, minAvailable = replicas − 1. Natija: dev 8, prod 9 obyekt.
5. {(kind, name): obyekt} lug'ati. Ingress port name yoki number bo'yicha Service.spec.ports dan biriga mos kelishi kerak; targetPort esa konteyner port nomi yoki raqami.

```
ob = {(h["kind"], h["metadata"]["name"]): h for h in hujjatlar}
```

6. mcpu() va mib() yordamchilari birliklarni bir xil o'lchovga keltiradi. requests > limits → K03 FAIL.
7. safe_dump_all → safe_load_all aylanmasi YAML'ga o'tmaydigan Python turlari (masalan, tuple) yo'qligini ko'rsatadi. Buzilganda 7 qoida yiqilishi kerak.

Bajargach o‘ylang: Namunadagi tekshir() bilan solishtiring: qaysi qoidalar faqat bitta obyektga, qaysilari bir nechta obyekt orasidagi moslikka tegishli? kubeconform yoki kube-linter bu ishning qaysi qismini bajara olmaydi?

5. 5-qadam. Versiyani chiqarish: RollingUpdate simulyatsiyasi va haqiqiy so'rovlar bilan canary

1. Sig'im talabdan kam bo'lsa yo'qotish boshlanadi: 5 × 50 = 250 < 260. Demak bitta Pod ham kam bo'lsa — yo'qotish bor.
2. Avval start tugagan Pod'lar tayyor bo'ladi; keyin eski Pod mavjudlik ≥ N − unav saqlansa o'chiriladi; keyin jami ≤ N + surge bo'lguncha yangi Pod'lar yaratiladi.
3. Recreate: eski Pod'lar birdaniga 0 ga tushadi, yangilari 30 s start oladi. Blue/green: yo'qotish 0, lekin maksimal Pod soni 12 — ikki baravar resurs.
4. Recreate: 30 s × 260 = 7800. maxUnavailable=1: har bosqichda 30 s davomida 10 so'rov/s yetishmaydi (6 bosqich × 30 × 10).
5. rng.random()·100 < w bo'lsa v2. Kam namunada xato ulushi va p95 juda shovqinli — 8 % xatoni 0,5 % dan ishonchli ajratib bo'lmaydi.

```
ver = "v2" if rng.random() * 100 < w else "v1"
```

6. Xato: v2 > v1 + 2 p.p.; kechikish: p95_v2 > max(1,5·p95_v1, p95_v1 + 2 ms). Juda kichik p95 da 1,5× tasodifiy shovqindan ham oshib ketadi.

```
np.percentile(latlar, 95)
```

7. Kutilgan: nuqsonli va sekin v2 5 % bosqichdayoq orqaga qaytariladi. 100 % bosqichda v1 namunasi yo'q — baza oldingi bosqichdan olinadi.

Bajargach o‘ylang: Namunadagi rolling() bilan solishtiring: eski Pod'larni o'chirish va yangilarini yaratish tartibini almashtirsangiz natija o'zgaradimi? Argo Rollouts canary tahlilida yana qaysi metrikalarni qo'shardingiz?

6. 6-qadam. HPA (autoscaling/v2) + Cluster Autoscaler + Erlang-C: yuklama simulyatsiyasi

1. ceil(4 × 90/70) = 6. |nisbat − 1| ≤ 0,1 bo'lsa joriy son saqlanadi. Bir nechta metrika bo'lsa — tavsiyalar maksimumi olinadi.

2. cpu_util = 100·min(λ/tayyor·XIZMAT, 1) — Pod 100 % dan ortiq ishlay olmaydi, shuning uchun HPA bir qadamda ~1,4× o'sadi. RPS (λ/tayyor) esa haqiqiy talabni ko'radi.
3. scaleUp: 15 s da max(joriy, 4) gacha qo'shish. scaleDown: oxirgi 300 s dagi eng katta tavsiyadan pastga tushmaslik va 60 s da ≤ 10 % kamaytirish.
4. Pending. ceil(pending / 3) tugun kerak (yo'ldagilarni ayiring), tugun_max dan oshmasin; tugun 120 s da tayyor. Ortiqcha tugun 600 s ketma-ket bo'lsa olib tashlanadi.
5. a = λ/μ, ρ = λ/(cμ). C = [a^c/c!/(1−ρ)] / [Σ_k<c a^k/k! + a^c/c!/(1−ρ)]. Kutish p95 ≈ ln(C/0,05)/(cμ − λ) (C ≤ 0,05 bo'lsa 0) + xizmat p95 ≈ 3·20 ms.

   ```
   a = lam / mu; rho = lam / (c * mu)
   ```

6. 4 tugun × 3 Pod × 50 = 600 so'rov/s, sakrashda talab ~770 so'rov/s — 10 daqiqa davomida ~170 so'rov/s yetishmaydi. Bulut CA bilan buzilish ~1,5–5 daqiqa.

Bajargach o‘ylang: Namunadagi sim() bilan solishtiring: siz Pending Pod'larni scaleDown paytida birinchi bo'lib bekor qildingizmi? «Oldindan 6 tugun, min 8» varianti nega eng kam buzilish beradi va uning narxi qanday?

7. 7-qadam. Operatsion hisobot: SLO, xato byudjeti, burn-rate ogohlantirishlari va Go/No-Go

1. Byudjet = (1 − SLO) × jami so'rovlar — ruxsat etilgan xatolar soni. Sarf = jami xato / byudjet; ~84 % kutiladi.
2. Burn rate = oynadagi xato ulushi / (1 − SLO). 1x — byudjet aynan 30 kunda tugaydi; 14,4x da 1 soatda byudjetning 2 % i yonadi (30 kun / 14,4 ≈ 2 kun).
3. Kumulyativ yig'indi: oyna yig'indisi = cs[i] − cs[i − oyna]. Boshida 0 qo'shing va indeksni 0 dan pastga tushirmang.

   ```
   cs = np.cumsum(np.r_[0, sorov])
   ```

4. Uzun oyna — ahamiyatlilik, qisqa oyna — muammo hozir ham davom etyaptimi. Faqat uzun oynada ogohlantirish tuzatilgandan keyin ham soatlab yonib turadi.
5. Insident boshidan keyingi oralikdagi birinchi True indeks (np.flatnonzero). Keskin — 5 daqiqada; sekin — faqat 6 soatlik va 3 kunlik qoidalarda.
6. kind, alert/expr/for/labels/annotations, severity ∈ {page, ticket}, qavslar balansi, 'for' = raqam + s/m/h, takroriy nomlar. Natija: VALID (5 qoida). Tungi chaqiruvda navbatchi nima qilishini bilishi kerak.
7. np.polyfit(kunlar, cho'qqi, 1) → a (o'sish/kun), b. Qoldi = (SIGIM − b)/a − 29; ~45 kun kutiladi. Barcha bandlar True bo'lsa — GO.

   ```
   a, b = np.polyfit(np.arange(30), cho, 1)
   ```

Bajargach o‘ylang: Namunadagi burn() va qoidalarni o'zingiznikiga solishtiring. Agar SLO 99,99 % qilinsa, xuddi shu ikki insident byudjetga qanday ta'sir qiladi va qaror o'zgaradimi?

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Xizmatga yangi endpoint va testlar** 1-qadam xizmatiga /v1/explain endpointini qo'shing (v1 logistik regressiya uchun eng katta hissa qo'shgan 5 ta belgi va ularning koeffitsient x qiymati). Kamida 3 ta yangi test yozing (200, 401, 422) va test jadvalini topshiring.	oson	5
2	**Konfiguratsiya sxemasini kengaytirish** 2-qadam sxemasiga 3 ta yangi kalit qo'shing (masalan, redis_url, cache_ttl_s, cors_origins) va har biri uchun tur/diapazon tekshiruvi yozing. To'g'ri va xato konfiguratsiyalar uchun natijalarni hamda manba (standart/ConfigMap/env) jadvalini ko'rsating.	oson	5
3	**Dockerfile qoidalari va obraz hajmi** 3-qadam tahlilchisiga 3 ta qoida qo'shing (.dockerignore borligi, LABEL versiyasi, HEALTHCHECK'da curl o'rniga python yoki wget). Paketlar hajmini taxminiy baholab (python:3.12-slim ~ 130 MB, numpy, scikit-learn, fastapi), bir bosqichli va ko'p bosqichli obraz hajmini jadvalda solishtiring.	oson	5
4	**Probe parametrlarini optimallashtirish** 2-qadam kubelet simulyatorida model yuklanish vaqti 20, 45, 90, 180 s bo'lgan 4 ssenariy uchun startupProbe (period, failureThreshold) ni tanlang: tayyor bo'lish vaqti minimal, restart 0 bo'lsin, osilib qolgan konteyner esa 5 daqiqada qayta ishga tushsin. Natijani jadval va tavsiya ko'rinishida bering.	o'rta	10
5	**Manifest validatorini kengaytirish** 4-qadam tekshiruvchisiga 5 ta yangi qoida qo'shing: NetworkPolicy podSelector Deployment'ga mos, Ingress TLS secretName mavjud (Secret obyekti), HPA metrikasi uchun requests.cpu bor, imagePullPolicy IfNotPresent/Always, PVC readOnly mount. Har qoida uchun uni buzadigan test manifest yozing va natijalar jadvalini topshiring.	o'rta	10
6	**Rollout strategiyasi tanlovi** 5-qadam rolling simulyatorida 10 nusxa, Pod start 60 s, talab 80 % sig'im sharoitida maxSurge (0..3) x maxUnavailable (0..3) setkasini hisoblang: davomiylik, yo'qolgan so'rovlar va qo'shimcha Pod-soniya. Pareto-optimal kombinatsiyalarni ajrating va grafikda ko'rsating.	o'rta	10
7	**Canary statistik kuchi** 5-qadam canary tahliliga ikki ulush uchun z-test qo'shing. v2 xato ulushi 1, 2, 4, 8 % bo'lganda (baza 0,5 %) 5 % bosqichda nuqsonni aniqlash uchun kerakli canary namunalari sonini va umumiy so'rovlar sonini toping; 20 ta urug' bilan takrorlab, aniqlash ehtimolini jadvalda bering.	o'rta	10
8	**HPA va CA sozlash tadqiqoti** 6-qadam simulyatorida o'z yuklama profilingizni (masalan, ertalabki cho'qqi + 2 ta sakrash) bering va kamida 8 konfiguratsiyani (nishon %, min/max, RPS metrikasi, tugun start vaqti, oldindan zaxira, scaleDown oynasi) SLO buzilishi va narx (Pod-	qiyin	15

	soat x narx + tugun-soat x narx) bo'yicha solishtiring. Pareto-chegara grafigini chizing va tanlovni asoslang.		
9	**To'liq joylashtirish to'plami** 1-4 qadamlar asosida bitta skript yozing: u xizmat kodini (main.py), requirements.txt, Dockerfile, docker-compose.yml va dev/prod K8s manifestlarini /tmp/deploy papkasiga generatsiya qilsin, barcha validatorlarni ishga tushirsin va umumiy PASS/FAIL hisobotini chiqarsin. Agar real Docker yoki Kubernetes (minikube, kind, k3s) bo'lsa, docker build va kubectl apply natijasini skrinshot bilan ilova qiling; bo'lmasa simulyatsiya natijalarini bering.	qiyin	15
10	**Operatsion hisobot o'z xizmatingiz uchun** O'z AI xizmatingiz (masalan, tasvir tasnifi, chatbot, tavsiya tizimi) uchun 7-qadamni moslang: SLO'larni (mavjudlik va kechikish) tanlang, 30 kunlik sintetik ma'lumotga 3 ta turli insident kiriting, burn-rate qoidalari va PrometheusRule'ni generatsiya qiling, aniqlash kechikishlarini jadvalda bering, sig'im prognozi va Go/No-Go qarorini raqamlar bilan asoslang (kamida 400 so'z).	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: qandli diabet xavfini baholash API oson

Soha: Tibbiyot

Poliklinikalar tarmog'i bemor laboratoriya ko'rsatkichlari bo'yicha diabet xavfini baholovchi ichki API'ni Kubernetes'ga joylashtiradi. Ish vaqtida 30 so'rov/s, kechasi deyarli nol. Ma'lumot shifoxona tarmog'idan chiqmaydi, model har chorakda yangilanadi.

Model	sklearn diabetes (regressiya -> xavf darajasi), Ridge v1 / GBR v2
Yuklama profili	kunduz 30, tun 2 so'rov/s
SLO	p95 <= 150 ms, 99,5 %
Klaster	3 tugun on-prem
Rollout	RollingUpdate maxSurge 1
HPA	min 2, max 6, CPU 60 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sklearn diabetes (regressiya -> xavf darajasi), Ridge v1 / GBR v2): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 6, CPU 60 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxSurge 1) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

2-variant. Agro: tuproq namligi bo'yicha sug'orish tavsiyasi oson

Soha: Qishloq xo'jaligi

Fermer xo'jaliklari IoT sensorlari har 10 daqiqada ma'lumot yuboradi, xizmat sug'orish kerakmi-yo'qligini qaytaradi. Sensorlar soni mavsumda 2000 dan 15000 gacha o'sadi. Xizmat tuman data-markazidagi kichik k3s klasterida ishlaydi.

Model	make_regression (8 belgi) -> RandomForest
Yuklama profili	2000 -> 15000 qurilma / 10 daq
SLO	p95 <= 500 ms, 99 %
Klaster	k3s, 3 tugun
Rollout	Recreate vs RollingUpdate
HPA	min 1, max 4, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_regression (8 belgi) -> RandomForest): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 1, max 4, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (Recreate vs RollingUpdate) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

3-variant. Transport: taksi narxini dinamik hisoblash oson

Soha: Transport

Taksi agregatori har buyurtmada narxni ML model bilan hisoblaydi; juma kechqurun yuklama 5 barobar oshadi. Kechikish ilova tajribasiga bevosita ta'sir qiladi. Siz canary bilan yangi narx modelini xavfsiz chiqarasiz.

Model	make_regression (12 belgi) -> GradientBoosting
Yuklama profili	80 -> 400 so'rov/s (juma)
SLO	p99 <= 100 ms, 99,9 %
Klaster	bulut, 4 -> 12 tugun
Rollout	canary 5/25/50/100
HPA	min 4, max 30, CPU 65 % + RPS

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_regression (12 belgi) -> GradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 30, CPU 65 % + RPS, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 5/25/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

4-variant. Telekom: abonent ketishini (churn) bashorat qilish oson

Soha: Telekom

Operator CRM tizimi har abonent qo'ng'irog'ida churn ehtimolini so'raydi; call-markaz ish vaqtida 120 so'rov/s. Model har oy qayta o'qitiladi va A/B sinovdan o'tkaziladi. Konfiguratsiya ConfigMap orqali, API kalit Secret orqali boshqariladi.

Model	make_classification (20 belgi, nomutanosib 9:1) -> LogisticRegression
Yuklama profili	120 so'rov/s, 8:00-20:00
SLO	p95 <= 200 ms, 99,9 %
Klaster	on-prem, 5 tugun
Rollout	blue/green
HPA	min 3, max 10, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (20 belgi, nomutanosib 9:1) -> LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 10, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (blue/green) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

5-variant. Energetika: elektr iste'molini soatlik prognoz qilish oson

Soha: Energetika

Hududiy elektr tarmog'i kompaniyasi har soat 1200 podstansiya bo'yicha keyingi 24 soat iste'molini prognoz qiladi. Yuklama soat boshida qisqa cho'qqi beradi (paketli so'rovlar). Xizmat batch endpointi bilan ishlaydi.

Model	sintetik vaqt qatori (sinus + shovqin) -> Ridge, batch 64
Yuklama profili	soat boshida 1200 so'rov 2 daqiqada
SLO	batch p95 <= 2 s
Klaster	on-prem, 3 tugun
Rollout	RollingUpdate maxUnavailable 0
HPA	min 2, max 8, RPS 20/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik vaqt qatori (sinus + shovqin) -> Ridge, batch 64): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 8, RPS 20/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxUnavailable 0) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

6-variant. Moliya: kredit skoring xizmati oson

Soha: Moliya (bank)

Bank onlayn kredit arizalarini 2 soniyada ko'rib chiqishi kerak; skoring modeli regulyator talabiga ko'ra versiyalanadi va har javob model versiyasi bilan jurnallanadi. Xizmat ikki namespace'da (dev, prod) va NetworkPolicy bilan izolyatsiyalangan.

Model	make_classification (15 belgi) -> LogisticRegression v1 / RandomForest v2
Yuklama profili	25 so'rov/s, cho'qqi 90
SLO	p95 <= 300 ms, 99,95 %
Klaster	on-prem, 3 control-plane + 4 worker
Rollout	canary + qo'lda tasdiq
HPA	min 3, max 8, CPU 60 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (15 belgi) -> LogisticRegression v1 / RandomForest v2): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 8, CPU 60 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary + qo'lda tasdiq) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

7-variant. Ta'lim: insho baholash yordamchisi oson

Soha: Ta'lim

Universitet talabalar inshosini matn belgilari (uzunlik, so'z boyligi, xatolar soni) bo'yicha dastlabki baholovchi xizmatni ishga tushiradi. Sessiya davrida yuklama 10 barobar o'sadi. Xizmat docker-compose bilan sinovdan o'tib, keyin K8s'ga ko'chiriladi.

Model	sintetik matn belgilari (10 belgi) -> Ridge
Yuklama profili	5 -> 50 so'rov/s (sessiya)
SLO	p95 <= 1 s, 99 %
Klaster	docker-compose -> k3s 2 tugun
Rollout	RollingUpdate
HPA	min 1, max 5, CPU 75 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn belgilari (10 belgi) -> Ridge): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 1, max 5, CPU 75 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

8-variant. Sanoat: nasoslarning nosozligini vibratsiya bo'yicha bashorat qilish

o'rta

Soha: Sanoat

Kimyo zavodida 600 nasos sensori har 5 s da vibratsiya spektri xususiyatlarini yuboradi; xizmat nosozlik ehtimolini qaytaradi. Zavod tarmog'i internetdan uzilgan, obrazlar ichki registrdan (NAS) olinadi. Yangilanish faqat ta'mirlash oynasida.

Model	sintetik sensor (16 belgi) -> IsolationForest + LogisticRegression
Yuklama profili	120 so'rov/s doimiy
SLO	p99 <= 250 ms, 99,9 %
Klaster	havo bo'shlig'i (air-gap), 3 tugun
Rollout	ta'mirlash oynasida Rolling
HPA	min 3, max 6, CPU 60 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik sensor (16 belgi) -> IsolationForest + LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 6, CPU 60 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (ta'mirlash oynasida Rolling) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (120 so'rov/s doimiy, havo bo'shlig'i (air-gap), 3 tugun) — kamida 4 konfiguratsiya, SLO (p99 <= 250 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

9-variant. Xavfsizlik: kirish tizimida yuz embeddingini solishtirish o'rta

Soha: Xavfsizlik

Korxona turniketlarida xodimning yuz embeddingi (128 o'lchamli) bazadagi 5000 ta bilan solishtiriladi. Smena boshida 15 daqiqada 3000 kirish. Xizmat ishdan chiqsa turniket ochilmaydi, shuning uchun PDB va topologiya tarqalishi majburiy.

Model	sintetik embedding (128) -> kNN (sklearn NearestNeighbors)
Yuklama profili	smena boshida 200 so'rov/s, 15 daq
SLO	p99 <= 300 ms, 99,99 %
Klaster	on-prem, 3 tugun, 2 zona
Rollout	RollingUpdate maxSurge 2
HPA	min 4, max 12, RPS 25/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik embedding (128) -> kNN (sklearn NearestNeighbors)): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 12, RPS 25/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxSurge 2) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (smena boshida 200 so'rov/s, 15 daq, on-prem, 3 tugun, 2 zona) — kamida 4 konfiguratsiya, SLO (p99 <= 300 ms, 99,99 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

10-variant. Chakana savdo: mahsulot tavsiya API o'rta

Soha: Chakana savdo

Onlayn do'kon har sahifa ochilganda tavsiya ro'yxatini so'raydi; aksiya kunlari yuklama 8 barobar o'sadi. Natijalar Redis keshida 60 s saqlanadi. Siz compose'da kesh bilan, K8s'da esa HPA va Cluster Autoscaler bilan sinaysiz.

Model	sintetik foydalanuvchi-mahsulot matritsasi -> TruncatedSVD
Yuklama profili	150 -> 1200 so'rov/s (aksiya)
SLO	p95 <= 120 ms, 99,9 %
Klaster	bulut, 3 -> 15 tugun
Rollout	canary 10/50/100
HPA	min 3, max 40, CPU 60 % + RPS

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik foydalanuvchi-mahsulot matritsasi -> TruncatedSVD): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 40, CPU 60 % + RPS, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 10/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (150 -> 1200 so'rov/s (aksiya), bulut, 3 -> 15 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 120 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

11-variant. Logistika: yetkazib berish vaqtini bashorat qilish (ETA) o'rta

Soha: Logistika

Kuryerlik xizmati har buyurtmada ETA ni ML bilan hisoblaydi va mijozga ko'rsatadi; tushlik va kechki cho'qqilar. Model har hafta yangilanadi, xato bashorat mijoz shikoyatiga olib keladi — canary tahlilida MAE ham kuzatiladi.

Model	make_regression (10 belgi, shovqinli) -> HistGradientBoosting
Yuklama profili	40 -> 250 so'rov/s (2 cho'qqi)
SLO	p95 <= 200 ms, 99,9 %
Klaster	bulut, 3 -> 8 tugun
Rollout	canary + MAE tahlili
HPA	min 3, max 15, CPU 65 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_regression (10 belgi, shovqinli) -> HistGradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 15, CPU 65 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary + MAE tahlili) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (40 -> 250 so'rov/s (2 cho'qqi), bulut, 3 -> 8 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 200 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

12-variant. Shahar: murojaatlarni avtomatik tasniflash (311 xizmati) o'rta

Soha: Shahar boshqaruvi

Hokimiyat portaliga kelgan fuqarolar murojaatlari (matn belgilari) tegishli bo'limga yo'naltiriladi. Ish kunlari 9:00-11:00 cho'qqi. Xizmat davlat data-markazidagi klasterda, barcha sirlar Secret orqali, tasvirlar root'siz.

Model	sintetik matn TF-IDF (500 belgi) -> LinearSVC
Yuklama profili	10 -> 60 so'rov/s
SLO	p95 <= 400 ms, 99,5 %
Klaster	on-prem, 4 tugun
Rollout	RollingUpdate maxSurge 25 %
HPA	min 2, max 8, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn TF-IDF (500 belgi) -> LinearSVC): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 8, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate maxSurge 25 %) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (10 -> 60 so'rov/s, on-prem, 4 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 400 ms, 99,5 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

13-variant. Ekologiya: suv sifati anomaliyasini aniqlash o'rta

Soha: Ekologiya

Suv ta'minoti korxonasi 80 nazorat nuqtasidan har daqiqada pH, loyqalik, xlor ma'lumotini oladi; anomaliya bo'lsa navbatchiga ogohlantirish. Xizmat kichik edge-klasterda (3 ta mini-PC) ishlaydi, elektr uzilishiga UPS bilan chidamli bo'lishi kerak.

Model	sintetik sensor (6 belgi) -> IsolationForest
Yuklama profili	80 so'rov/daq doimiy
SLO	p95 <= 1 s, 99,9 %
Klaster	edge k3s, 3 mini-PC
Rollout	Recreate taqiqlangan, Rolling
HPA	min 2, max 3, CPU 80 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik sensor (6 belgi) -> IsolationForest): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 3, CPU 80 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (Recreate taqiqlangan, Rolling) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (80 so'rov/daq doimiy, edge k3s, 3 mini-PC) — kamida 4 konfiguratsiya, SLO (p95 <= 1 s, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

14-variant. Sport: o'yinchi jarohati xavfini baholash o'rta

Soha: Sport

Futbol akademiyasi mashg'ulot yuklamasi (GPS, yurak urishi) bo'yicha o'yinchi jarohati xavfini har kuni hisoblaydi; murabbiylar planshetdan so'raydi. Yuklama kichik, lekin mavjudlik mashg'ulot vaqtida muhim. Xizmat bitta tugunli klasterdan 3 tugunliga ko'chiriladi.

Model	make_classification (12 belgi) -> RandomForest
Yuklama profili	2 -> 15 so'rov/s
SLO	p95 <= 500 ms, 99,5 %
Klaster	1 -> 3 tugun
Rollout	blue/green
HPA	min 1, max 3, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (12 belgi) -> RandomForest): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 1, max 3, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (blue/green) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (2 -> 15 so'rov/s, 1 -> 3 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 500 ms, 99,5 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

15-variant. Media: kontent moderatsiyasi (toksik izohlar) o'rta

Soha: Media

Yangiliklar portali izohlarni chop etishdan oldin toksiklikka tekshiradi; mashhur yangilikda bir necha daqiqada yuklama 20 barobar sakraydi. Xizmat bulutda, Cluster Autoscaler tugun qo'shishi 3 daqiqa oladi — oldindan zaxira siyosatini tanlaysiz.

Model	sintetik matn belgilari (300) -> LogisticRegression
Yuklama profili	20 -> 400 so'rov/s (10 daq)
SLO	p95 <= 250 ms, 99,9 %
Klaster	bulut, 2 -> 12 tugun
Rollout	canary 5/20/100
HPA	min 2, max 40, RPS 30/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn belgilari (300) -> LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 40, RPS 30/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 5/20/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (20 -> 400 so'rov/s (10 daq), bulut, 2 -> 12 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 250 ms, 99,9 %) buzilishi va Pod/tugun-soat.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

16-variant. Tibbiyot: tez yordam chaqiruvlarini triaj qilish qiyin

Soha: Tibbiyot (shoshilinch yordam)

103 xizmati operatori chaqiruv belgilarini kiritadi, model ustuvorlik darajasini (1-4) taklif qiladi. Xizmat to'xtashi mumkin emas: ikki klaster (asosiy va zaxira), har yangilanish canary orqali, xato byudjeti juda kichik. Burn-rate ogohlantirishlari pager'ga ulanadi.

Model	make_classification (4 sinf, 18 belgi) -> GradientBoosting
Yuklama profili	15 -> 60 so'rov/s
SLO	p99 <= 300 ms, 99,99 %
Klaster	2 klaster x 3 tugun
Rollout	canary 1/10/50/100
HPA	min 4, max 10, CPU 50 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (4 sinf, 18 belgi) -> GradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 10, CPU 50 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 1/10/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (15 -> 60 so'rov/s, 2 klaster x 3 tugun) — kamida 4 konfiguratsiya, SLO (p99 <= 300 ms, 99,99 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

17-variant. Neft-gaz: quduq debitini bashorat qilish qiyin

Soha: Neft-gaz

Kompaniya 900 quduq bo'yicha har soat debit prognozini hisoblaydi va dispetcherga ko'rsatadi. Klaster dala ofisida, internet sun'iy yo'ldosh orqali sekin, shuning uchun obrazlar kichik bo'lishi va ichki registrda keshlanishi shart. Dockerfile hajmini optimallashtirasiz.

Model	sintetik vaqt qatori (14 belgi) -> HistGradientBoosting
Yuklama profili	900 so'rov / soat boshida
SLO	batch p95 <= 3 s, 99,5 %
Klaster	dala ofisi, 3 tugun, sekin WAN
Rollout	RollingUpdate, obraz oldindan tortiladi
HPA	min 2, max 4, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik vaqt qatori (14 belgi) -> HistGradientBoosting): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 4, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate, obraz oldindan tortiladi) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (900 so'rov / soat boshida, dala ofisi, 3 tugun, sekin WAN) — kamida 4 konfiguratsiya, SLO (batch p95 <= 3 s, 99,5 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

18-variant. Sug'urta: avtomobil zarari da'vosini firibgarlikka tekshirish qiyin

Soha: Sug'urta

Sug'urta kompaniyasi har da'voni firibgarlik ehtimoli bo'yicha baholaydi; da'volar ish kunlari tekis keladi, oy oxirida 3 barobar. Regulyator har model versiyasining audit izini talab qiladi: x-model-version, sha256 va konfiguratsiya xeshi jurnallanadi.

Model	make_classification (25 belgi, 19:1) -> RandomForest v1 / GBC v2
Yuklama profili	10 -> 30 so'rov/s (oy oxiri)
SLO	p95 <= 800 ms, 99,9 %
Klaster	on-prem, 3 control-plane + 3 worker
Rollout	canary + audit
HPA	min 2, max 6, CPU 65 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: make_classification (25 belgi, 19:1) -> RandomForest v1 / GBC v2): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 2, max 6, CPU 65 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary + audit) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (10 -> 30 so'rov/s (oy oxiri), on-prem, 3 control-plane + 3 worker) — kamida 4 konfiguratsiya, SLO (p95 <= 800 ms, 99,9 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

19-variant. Davlat xizmatlari: hujjat turini tasniflash (OCR natijasi) qiyin

Soha: Davlat xizmatlari

Yagona interaktiv portalda yuklangan skanlar OCR'dan keyin matn belgilari bo'yicha hujjat turiga (pasport, diplom, ma'lumotnoma ...) ajratiladi. Murojaatlar soni yil davomida o'sib bormoqda — sig'im prognozi va HPA maxReplicas rejasini tuzasiz.

Model	sintetik matn belgilari (200) -> MultinomialNB / LogisticRegression
Yuklama profili	50 so'rov/s, oyiga +5 %
SLO	p95 <= 600 ms, 99,9 %
Klaster	davlat DC, 6 tugun
Rollout	RollingUpdate + canary
HPA	min 3, max 20, CPU 70 %

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik matn belgilari (200) -> MultinomialNB / LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 3, max 20, CPU 70 %, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (RollingUpdate + canary) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (50 so'rov/s, oyiga +5 %, davlat DC, 6 tugun) — kamida 4 konfiguratsiya, SLO (p95 <= 600 ms, 99,9 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20

20-variant. Kiberxavfsizlik: login urinishlarida anomaliyani aniqlash qiyin

Soha: Kiberxavfsizlik

Korporativ SSO tizimi har login urinishini (IP, vaqt, qurilma belgilari) xavf bali bo'yicha baholaydi; hujum paytida yuklama 50 barobar sakraydi va aynan shu payt xizmat ishlashi shart. Rate-limit, NetworkPolicy va tez masshtablash muhim.

Model	sintetik log belgilari (10) -> IsolationForest + LogisticRegression
Yuklama profili	50 -> 2500 so'rov/s (hujum)
SLO	p99 <= 150 ms, 99,95 %
Klaster	bulut, 4 -> 20 tugun
Rollout	canary 5/50/100
HPA	min 4, max 60, RPS 40/Pod

QADAMLAR

1. 1-qadam kabi FastAPI xizmatini yozing (model: sintetik log belgilari (10) -> IsolationForest + LogisticRegression): probe'lar, API kalit, so'rov ID, batch, /metrics; kamida 10 ta TestClient testi PASS.
2. 2-qadam: konfiguratsiya sxemasi (standart < ConfigMap < env) va chuqur readiness; variant model yuklanish vaqti uchun probe parametrlarini kubelet simulyatorida tekshiring.
3. 3-qadam: ko'p bosqichli Dockerfile va docker-compose (xizmat + kerakli yordamchi xizmatlar) generatsiyasi, 13 qoida va compose validatsiyasi PASS; qatlam keshi tahlili.
4. 4-qadam: dev/prod manifestlari (Deployment, Service, HPA min 4, max 60, RPS 40/Pod, ConfigMap, Secret, Ingress, PDB, NetworkPolicy) — 18 qoida PASS, buzilgan nusxada xatolar ushlansin.
5. 5-qadam: rollout (canary 5/50/100) simulyatsiyasi va TestClient bilan canary — sog'lom va nuqsonli v2 uchun qarorlar jadvali.
6. 6-qadam: HPA + Cluster Autoscaler simulyatsiyasi (50 -> 2500 so'rov/s (hujum), bulut, 4 -> 20 tugun) — kamida 4 konfiguratsiya, SLO (p99 <= 150 ms, 99,95 %) buzilishi va Pod/tugun-soat.
7. 7-qadam: 30 kunlik sintetik ma'lumotda SLO va xato byudjeti, burn-rate qoidalari, PrometheusRule YAML, sig'im prognozi va Go/No-Go qarori.

Topshiriladi: Kod (.py: xizmat, generatorlar, simulyatorlar); Dockerfile, docker-compose.yml va K8s YAML (PASS natijasi bilan); Natijalar jadvallari va grafiklar (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatsion hisobot: SLO, ogohlantirishlar va Go/No-Go

Baholash (100 ball): To'g'ri ishlaydigan kod va testlar — 30, Manifestlar va validatsiya — 25, Simulyatsiya va tahlil — 25, Hisobot sifati — 20